



تقرير مشروع البرمجة التفرعية

High-Performance E-Commerce Backend Engine

(NFR1 - NFR5) أول 5 طلبات

إشراف المهندس :

حذيفة عقيل

ما تم تنفيذه	الطلب	الطالب
Race Condition — SELECT FOR UPDATE + Optimistic Locking + Redis Distributed Locks	NFR1	منير لؤي المزيك
Resource Management — Bounded Thread/Connection Pools + Gunicorn/Celery Tuning	NFR2	معاذ محمد بشار زكريا
Async Queue — Celery + Redis Broker + Idempotent Tasks + on_commit Dispatch	NFR3	غاردينيا ناصر فرح
Batch Processing — Chunked Iterator + Parallel Executor + Daily Sales Aggregator	NFR4	آية غسان بدران
Load Distribution — Nginx least_conn + 3 App Instances + Stateless Sessions	NFR5	عمر حسن برهم

المشكلة

عند وجود طلبات متزامنة لشراء نفس المنتج، قد تقرأ كل عملية نفس قيمة المخزون وتفترض توفره، مما يؤدي إلى بيع كميات أكثر مما هو متاح (Overselling). هذه هي مشكلة Race Condition الكلاسيكية على الـ shared state.

الحلول المقترحة

- Pessimistic Locking — SELECT FOR UPDATE: يقفل السجل بالكامل حتى ينتهي الـ transaction
- Optimistic Locking — Versioning: يحفظ رقم إصدار ويرفض الكتابة إن تغير
- Distributed Lock — Redis SET NX: قفل موزع بين عدة Processes
- Atomic F() expressions: تحديث عداد بشكل Atomic بدون قراءة ثم كتابة

هل بنية المشروع تدعم هذه الحلول؟

المشروع مبني على Django + PostgreSQL الذي يدعم SELECT FOR UPDATE بشكل كامل. Redis متوفر ومُكوّن. جميع النماذج تمتلك حقول version لـ Optimistic Locking. لذا فهو يدعم هذه الحلول

الحل المختار

- Pessimistic Locking (SELECT FOR UPDATE) كالحل الأساسي لجميع العمليات الحرجة، مع:
- Optimistic CAS في bump_version () لعمليات القراءة الخفيفة
 - Redis Distributed Lock للقفل عبر Processes متعددة
 - ترتيب القفل بـ PK ASC في bulk_reserve () لمنع Deadlock
 - F() expressions لتحديث loyalty points بشكل Atomic

لماذا هذا الحل؟

SELECT FOR UPDATE هو الضمان الأقوى ضد Oversell في بيئة متعددة العمليات. ترتيب القفل بـ PK ASC يمنع الـ Circular Wait (شرط Deadlock). Redis Lock يحل المشكلة عندما يكون لدينا 3 App Instances خلف Nginx.

كيف تم الاختبار؟

- test_oversold_one_unit(): 10 Threads تتنافس على وحدة واحدة — النتيجة: نجاح واحد فقط
- test_concurrent_reservations_consistent(): 20 Thread على 5 وحدات — دقة عدد العمليات الناجحة
- test_bulk_reserve_no_deadlock(): قفل بترتيب عكسي من الـ Thread الثاني — لا Deadlock
- test_concurrency_payments.py: معالجة دفع متوازيات — نجاح واحد فقط
- اختبار JMeter: 100 طلب متزامن على endpoint الشراء

الملفات الرئيسية

core/concurrency/locks.py | apps/inventory/services.py | apps/orders/services.py |
apps/payments/services.py | tests/unit/test_concurrency_*.py

المشكلة

عند التحميل العالي، يمكن للنظام أن يستنزف الموارد (Threads، DB Connections، ذاكرة) مما يؤدي إلى انهيار الأداء أو crash كامل للتطبيق. المشكلة الثانية هي إهدار الموارد بتخصيص أكثر مما يلزم.

الحلول المقترحة

- Thread Pool مع حد أقصى (BoundedThreadPoolExecutor)
- Connection Pool لل DB بـ CONN_MAX_AGE
- Semaphore / Capacity Limiter لكل نوع عملية
- Gunicorn Workers + Threads tuning
- Celery Concurrency tuning

هل بنية المشروع تدعم هذه الحلول؟

نعم. المشروع يستخدم Gunicorn كـ WSGI server و Celery كـ Task Queue. كلاهما يدعم تكوين ال Workers وال PostgreSQL Concurrency. يدعم Connection Pooling.

الحل المختار

- طبقنا نظام إدارة موارد متكامل عبر `core/resources/pool.py`
- `CapacityLimitedThreadPoolExecutor`: يحد ال Threads المتوازية لكل نوع عملية
- `resource_slot() / capacity_limited()` decorators على كل service endpoint
- حدود مستقلة: `checkout=8, payment=8, batch=4`
- إعادة ال 503 عند تجاوز الحد مع metadata
- Celery: 4 concurrency، Gunicorn: 4 workers x 2 threads

لماذا هذا الحل؟

الحد المخصص لكل نوع عملية يضمن عدم تأثير عمليات ال Batch على أداء Checkout المستخدمين. إعادة ال 503 بشكل فوري أفضل من انتظار Request حتى يتوقف النظام. البنية تسمح بالتوليف بناء على قياسات حقيقية.

كيف تم الاختبار؟

- `test_resource_pool.py`: اختبارات وحدة شاملة لل pool mechanics
- `test_acquire_slot_rejects_when_full`: التحقق من رفض ال slot عند الامتلاء
- `test_bounded_executor_caps_parallel_work`: التحقق من حد ال workers
- اختبار Locust: Stress test بـ 50+ VUs مع مراقبة استهلاك الموارد
- Endpoint التشخيص: `GET /api/v1/_diag/pool` يعرض pool stats في الوقت الفعلي

الملفات الرئيسية:

core/resources/pool.py | config/settings/base.py | docker/entrypoint.sh |
core/diagnostics/views.py | tests/unit/test_resource_pool.py

المشكلة

بعض العمليات (إرسال الإيميل، توليد الفاتورة) تستغرق وقتاً ولا يجب أن تعطل استجابة ال HTTP. إذا انقطع الاتصال أو وقع السيرفر، هذه العمليات تضيع مع ال request. المشكلة الثانية: تكرار تنفيذ Task بسبب Retry مما يسبب إرسال إيميلات مكررة.

الفرق بين Async Execution و Async Queue

Async Execution (asyncio): تنفيذ غير متزامن داخل نفس العملية، لكن العملية تضيع عند انقطاع الاتصال.
 Async Queue (Celery + Redis): العملية محفوظة في Queue خارجي، تُنفَّذ حتى لو وقع السيرفر، ويمكن إعادة المحاولة تلقائياً.

نختار Queue لإرسال الفواتير لأن فقدانها يضر بالمستخدم. لل logs البسيطة asyncio كافٍ.

هل بنية المشروع تدعم هذه الحلول؟

نعم. Redis مُثَبَّت ومُكوّن Celery مُهيأً مع Beat scheduler و Flower للمراقبة. Django ORM يدعم transaction.on_commit () لضمان إرسال Task فقط عند نجاح ال transaction.

الحل المختار

- Message Queue ك Celery + Redis Broker
- task ACKS_LATE=True: لا Ack حتى ينتهي التنفيذ بنجاح (ضمان إعادة المحاولة)
- REJECT_ON_WORKER_LOST=True: إعادة ال task تلقائياً عند سقوط ال worker
- transaction.on_commit(): إرسال ال task فقط بعد ال commit ال DB لمنع race condition
- Idempotency: جدول OrderEmailDispatch يمنع إرسال إيميل مكرر
- autoretry_for على الأخطاء المؤقتة فقط (SMTPException, ConnectionError)

لماذا هذا الحل؟

ACKS_LATE + REJECT_ON_WORKER_LOST يضمنان At-Least-Once execution. جدول ال Idempotency يحولها إلى Exactly-Once من منظور المستخدم. on_commit يمنع إرسال إيميل ل order لم تُحفظ أصلاً.

كيف تم الاختبار؟

- تشغيل celery_worker + celery_beat + flower عبر docker-compose
- اختبار إنشاء Order وتتبع Task في Flower Dashboard
- محاكاة فشل SMTP والتحقق من Retry بـ max_retries=5
- اختبار Idempotency: تنفيذ Task مرتين لل Order نفسه — إيميل واحد فقط
- JMETER: 50 طلب شراء متزامن، التحقق من إرسال 50 إيميل بالضبط

الملفات الرئيسية:

config/celery.py | apps/tasks/notifications.py | apps/tasks/invoicing.py |
apps/orders/services.py | docker-compose.yml

المشكلة

تقرير المبيعات اليومي يحتاج معالجة ملايين سجلات OrderItems. إذا حمّلنا كل السجلات دفعة واحدة في الذاكرة، ستنفجر الـ RAM. إذا شغلنا المعالجة بشكل تسلسلي، ستستغرق ساعات. الحل يجب أن يكون memory-safe وسريعاً.

الحلول المقترحة

- جلب كل السجلات إلى الذاكرة ثم المعالجة (بطيء + خطر على الذاكرة)
- Streaming Cursor بـ chunking + iterator() (آمن على الذاكرة)
- MapReduce: تقسيم + معالجة متوازية + دمج النتائج
- Database-level aggregation بـ GROUP BY (سريع لكن محدود المرونة)

هل بنية المشروع تدعم هذه الحلول؟

نعم. Django ORM يدعم `queryset.iterator(chunk_size=N)` لـ `Streaming`. Celery + Python لـ `ThreadPoolExecutor` يسمحان بالمعالجة المتوازية. NFR2 Resource Pool يتحكم في عدد الـ `workers` حتى لا يؤثر الـ `Batch` على الـ `Checkout`.

الحل المختار

- نظام MapReduce مبسط عبر `core/batch/chunked.py`:
- `iter_in_chunks(): queryset.iterator(chunk_size=1000)` — لا يحمل كل الـ DB في الذاكرة
 - `process_in_parallel(): bounded_executor(resource='batch')` — معالجة متوازية بحد أقصى 4 `workers`
 - `DailySalesAggregator: Associative + Commutative merge` — يمكن دمج أي ترتيب
 - `DailySalesReport: update_or_create()` — `Idempotent` — يمكن إعادة التشغيل بأمان
 - `Celery Beat`: تشغيل تلقائي كل يوم الساعة 02:00 UTC

لماذا هذا الحل؟

`iterator()` يجعل الـ `peak memory` يعتمد على `chunk_size` لا على حجم الـ `dataset`. الـ `Bounded Executor` يضمن عدم تأثير الـ `Batch` على الـ `Idempotency`. `Foreground requests` تسمح بإعادة التشغيل عند الفشل.

كيف تم الاختبار؟

- `test_iter_in_chunks`: اختبار الـ `chunking` مع `datasets` مختلفة الأحجام
- `test_merge_is_associative/commutative`: ضمان صحة الـ `merge` بأي ترتيب
- `test_daily_sales_report_creation`: اختبار `end-to-end` مع `Idempotency`
- `test_process_in_parallel`: التحقق من اكتمال كل الـ `chunks`
- اختبار أداء: 10,000 سجل بـ `chunk_size=1000` و `max_workers=4`

الملفات الرئيسية:

core/batch/chunked.py | apps/tasks/daily_sales_batch.py | apps/orders/models.py
(DailySalesReport) | tests/unit/test_batch_chunked.py

المشكلة

Application Instance واحد لديه حد أقصى للطلبات التي يمكنه معالجتها. عند التحميل العالي أو عطل Instance واحد، النظام كله يتعطل. يجب توزيع الطلبات على عدة Instances مع ضمان ال Failover.

الحلول المقترحة

- Round Robin: توزيع دوري متساوٍ — بسيط لكن لا يراعي الحمل الفعلي
- Least Connections: الإرسال لل Instance الأقل انشغالاً — أذكى وأعدل
- IP Hash: نفس المستخدم دائماً لنفس ال Instance — يحل مشكلة ال Session، لكن يسبب عدم توازن
- Weighted: توزيع بأوزان مختلفة حسب قوة كل Instance

هل بنية المشروع تدعم هذه الحلول؟

نعم. المشروع محاط ب Docker Compose ويمكن تشغيل 3 Instances بسهولة. Nginx يدعم جميع استراتيجيات التوزيع. ال Sessions محفوظة في Redis لا في Process Memory، مما يجعل النظام Stateless.

الحل المختار

- Nginx Load Balancer ب least_conn strategy مع 3 App Instances:
- least_conn ب upstream django_backend: web1:8000, web2:8000, web3:8000
 - max_fails=3 fail_timeout=10s: Failover تلقائي عند فشل Instance
 - X-Served-By header: كل Response يحمل اسم ال Instance الذي عالجه
 - SESSION_ENGINE = 'cache': Sessions في Redis المشترك — لا Sticky Sessions
 - INSTANCE_ID: متغير بيئة مختلف لكل Instance لتحديد الهوية

لماذا least_conn وليس Round Robin؟

طلبات Checkout تستغرق أطول من طلبات قراءة المنتجات. Round Robin سيرسل طلبات جديدة ل Instance مشغول ب Checkout بينما Instance آخر خامل. least_conn يوزع بناء على الحمل الفعلي لا الترتيب الدوري.

كيف تم الاختبار؟

- تشغيل docker-compose up مع 3 Instances + Nginx
- 50 طلب متتالي وتتبع X-Served-By header في كل Response
- التحقق من التوزيع: كل Instance يستقبل ~33% من الطلبات
- docker-compose stop web1 والتحقق من استمرار الخدمة عبر web2 و web3
- JMeter: 100 VU متزامن، تحقق من توزيع الأحمال وعدم انقطاع الخدمة
- اختبار Histogram: مقارنة توزيع round_robin vs least_conn تحت حمل غير متساوٍ

الملفات الرئيسية:

docker/nginx.conf | docker-compose.yml | config/settings/base.py (INSTANCE_ID, SESSION_ENGINE) | core/aop/middleware.py (X-Instance-Id header)

NFR1 — Concurrent Access

- core/concurrency/locks.py — distributed_lock, select_for_update_or_skip, bump_version, with_optimistic_retry
- apps/inventory/services.py — reserve_stock, bulk_reserve (PK-ASC ordering)
- apps/inventory/models.py — StockItem (version field)
- apps/orders/services.py — place_order (atomic + bulk_reserve)
- apps/payments/services.py — capture_payment, process_webhook (idempotent)
- apps/users/services.py — adjust_loyalty_points (F()) expression
- tests/unit/test_concurrency_inventory.py — threading tests
- tests/unit/test_concurrency_payments.py — concurrent capture tests
- tests/unit/test_concurrency_loyalty.py — 50-thread loyalty test

NFR2 — Resource Management

- core/resources/pool.py — _ResourcePool, resource_slot, capacity_limited, CapacityLimitedThreadPoolExecutor
- core/diagnostics/views.py — PoolDiagnosticsView, InstanceView
- config/settings/base.py — RESOURCE_LIMITS, GUNICORN_*, CELERY_CONCURRENCY
- docker/entrypoint.sh — gunicorn/celery flags
- tests/unit/test_resource_pool.py — pool mechanics tests

NFR3 — Async Queue

- config/celery.py — Celery app configuration
- apps/tasks/notifications.py — send_order_confirmation (idempotent, autoretry)
- apps/tasks/invoicing.py — generate_invoice (idempotent, autoretry)
- apps/tasks/__init__.py — beat_schedule
- apps/orders/models.py — OrderEmailDispatch (idempotency table)
- docker-compose.yml — celery_worker, celery_beat, flower services

NFR4 — Batch Processing

- core/batch/chunked.py — iter_in_chunks, process_in_parallel, DailySalesAggregator
- apps/tasks/daily_sales_batch.py — run_daily_sales Celery task
- apps/orders/models.py — DailySalesReport model
- tests/unit/test_batch_chunked.py — chunking + aggregator + parallel tests

NFR5 — Load Distribution

- docker/nginx.conf — upstream least_conn, 3 backends, failover config
- docker-compose.yml — web1, web2, web3 services + nginx
- config/settings/base.py — SESSION_ENGINE=cache, INSTANCE_ID
- core/aop/middleware.py — X-Instance-Id header
- core/diagnostics/views.py — InstanceView endpoint

الطلب الأول : اختبار Race Condition / Data Integrity

الهدف: عدة مستخدمين يحاولون شراء نفس المنتج بنفس الوقت.
❖ قبل تطبيق الحل:

```
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> docker compose exec web1 python -m pytest tests/unit/test_concurrency_inventory.py::test_oversold_one_unit -q
F
[100%]
===== FAILURES =====
===== test_oversold_one_unit =====
stock_item = <StockItem: Stock<2: on_hand=10, reserved=0>>

def test_oversold_one_unit(stock_item):
    """One unit of stock, ten concurrent reservers -> exactly one wins."""
    StockItem.objects.filter(pk=stock_item.pk).update(on_hand=1, reserved=0)

    def reserve(i):
        services.reserve_stock(
            product_id=stock_item.product_id, qty=1, reference=f"req-{i}"
        )

    results = _run_concurrently(reserve, n_threads=10)
    successes = [r for r in results if r[0] == "ok"]
    failures = [r for r in results if r[0] == "err"]

> assert len(successes) == 1, f"expected exactly 1 success, got {len(successes)}"
E   AssertionError: expected exactly 1 success, got 10
E   assert 10 == 1
E   + where 10 = len([('ok', None), ('ok', None), ('ok', None), ('ok', None), ('ok', None), ('ok', None), ...])

tests/unit/test_concurrency_inventory.py:68: AssertionError
```

```
===== short test summary info =====
FAILED tests/unit/test_concurrency_inventory.py::test_oversold_one_unit - AssertionError: expected exactly 1 success, got 10
```

قبل تطبيق أقفال التزامن، نجحت 10 عمليات حجز متزامنة رغم أن المخزون يحتوي على قطعة واحدة فقط.
هذا يثبت وجود Race Condition لأن العمليات المتزامنة عدّلت نفس المورد بدون حماية كافية.

❖ بعد تطبيق الحل :

```
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> docker compose exec web1 python -m pytest tests/unit/test_concurrency_inventory.py::test_oversold_one_unit -q
.
[100%]
1 passed in 2.30s
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> |
```

بعد تطبيق أقفال التزامن على سجل المخزون، نجح طلب واحد فقط من أصل 10 طلبات متزامنة على منتج كميته قطعة واحدة، وفشلت باقي الطلبات بسبب نفاد المخزون. هذا يثبت معالجة مشكلة Race Condition وحماية البيانات المشتركة من التضارب.

الطلب الثاني : اختبار Resource Management

الهدف: معرفة هل النظام يضبط عدد العمليات المتوازية أو يتركها مفتوحة.
❖ قبل تطبيق الحل :

```
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> docker compose exec web1 python -m pytest tests/unit/test_resource_pool.py::test_bounded_executor_caps_parallel_work_to_resource_limit -q
F
===== FAILURES ===== [100%]
----- test_bounded_executor_caps_parallel_work_to_resource_limit -----

def test_bounded_executor_caps_parallel_work_to_resource_limit():
    resource = "unit_executor"
    max_seen = 0
    running = 0
    lock = threading.Lock()

    def work():
        nonlocal max_seen, running
        with lock:
            running += 1
            max_seen = max(max_seen, running)
            time.sleep(0.03)
        with lock:
            running -= 1
        return 1

    with _settings_for(resource, 2):
        with bounded_executor(max_workers=6, resource=resource) as executor:
            futures = [executor.submit(work) for _ in range(8)]
            assert sum(f.result(timeout=1) for f in futures) == 8

>         assert max_seen <= 2
E         assert 6 <= 2

tests/unit/test_resource_pool.py:135: AssertionError
===== short test summary info =====
FAILED tests/unit/test_resource_pool.py::test_bounded_executor_caps_parallel_work_to_resource_limit - assert 6 <= 2
```

قبل تطبيق إدارة الموارد، لم تكن هناك آلية فعالة لتقييد العمليات المتوازية. كان التنفيذ يسمح بتشغيل عدد كبير من المهام في نفس الوقت دون احترام حد الموارد. رغم ضبط الحد على 2 عمليات متزامنة، وصل عدد العمليات المتوازية إلى 6، مما يثبت غياب التحكم بالقدرة واحتمال استهلاك موارد النظام بشكل مفرط تحت الضغط.

❖ بعد تطبيق الحل:

```
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> docker compose exec web1 python -m pytest tests/unit/test_resource_pool.py::test_bounded_executor_caps_parallel_work_to_resource_limit -q
.
1 passed in 0.32s [100%]
```

وهذه صفحة مراقبة الموارد ب اسم Pool Diagnostics للإطلاع عليها :

[Pool Diagnostics – Django REST framework](#)

HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "instance_id": "web1",
  "outer_caps": {
    "gunicorn_workers": 4,
    "gunicorn_threads": 2,
    "gunicorn_worker_class": "sync",
    "gunicorn_timeout": 30,
    "celery_concurrency": 4
  },
  "resource_acquire_timeout_seconds": 1.0,
  "pools": {
    "batch": {
      "limit": 4,
      "in_flight": 0,
      "available": 4,
      "acquired_total": 0,
      "rejected_total": 0
    },
    "checkout": {
      "limit": 8,
      "in_flight": 0,
      "available": 8,
      "acquired_total": 0,
      "rejected_total": 0
    },
    "internal_pool": {
      "limit": 16,
      "in_flight": 0,
      "available": 16,
      "acquired_total": 0,
      "rejected_total": 0
    },
    "payment": {
      "limit": 8,
      "in_flight": 0,
      "available": 8,
      "acquired_total": 0,
      "rejected_total": 0
    }
  }
}
```

بعد تطبيق إدارة الموارد، أصبحت الموارد الحساسة في النظام مراقبة ومحددة بسقوف واضحة. تظهر واجهة Pool Diagnostics أن عمليات checkout محددة بـ 8 عمليات متزامنة، و payment بـ 8، و batch بـ 4، و internal_pool بـ 16. كما يوضح الحقل in_flight عدد العمليات الجارية حالياً، و rejected_total عدد العمليات التي تم رفضها عند امتلاء المورد. هذا يثبت أن النظام لا يسمح بعدد غير مضبوط من العمليات، بل يتحكم بالقدرة لمنع استهلاك الموارد بشكل مفرط تحت الضغط.

الهدف: تقارن زمن checkout قبل وبعد نقل invoice/email إلى queue.
أي الفاتورة والإشعار لا يجب أن يعملوا داخل طلب الشراء نفسه، بل نضعها بـ Queue مثل Celery/Redis،
والمستخدم يعيد له الرد بسرعة.
❖ قبل تطبيق الحل :

```
>> '@ | docker compose exec -T web1 python manage.py shell
audit.start
audit.start
audit.ok
timed
Starting invoice generation for order 4001
Invoice generated successfully for order 4001
Task tasks.invoicing.generate_invoice[7cd3e936-58d1-4ec6-bfa6-af3ccfecfc25] succeeded in 4.14998793399991s: None
Starting confirmation for order 4001
Order confirmation SENT for order 4001
Task tasks.notifications.send_order_confirmation[c7c2962c-2c0f-4fe5-947e-42dd94fb3f86] succeeded in 5.0175869540007625s:
None
audit.ok
timed
ORDER_ID=4001
ELAPSED_SECONDS=9.52
INVOICE_URL=https://storage.example.com/invoices/order-4001.pdf
EMAIL_STATUS=sent
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine>
```

قبل تطبيق الطوابير غير المتزامنة، كان طلب إتمام الشراء ينتظر توليد الفاتورة وإرسال الإشعار داخل نفس مسار الطلب. ظهرت النتيجة أن توليد الفاتورة استغرق حوالي 4.14 ثانية، وإرسال الإشعار حوالي 5.01 ثانية، ليصبح زمن استجابة الطلب كاملاً 9.52 ثانية. هذا يثبت أن المهام الخلفية لم تكن مفصولة عن المسار الرئيسي للمستخدم.

❖ بعد تطبيق الحل :

```
>> '@ | docker compose exec -T web1 python manage.py shell
audit.start
audit.start
audit.ok
timed
audit.ok
timed
ORDER_ID=2001
ELAPSED_SECONDS=0.44
INVOICE_URL_IMMEDIATE=None
EMAIL_EXISTS_IMMEDIATE=False
WAITING_12_SECONDS_FOR_CELERY...
INVOICE_URL_AFTER_WAIT=https://storage.example.com/invoices/order-2001.pdf
EMAIL_STATUS_AFTER_WAIT=sent
```

بعد تطبيق Asynchronous Queues باستخدام Celery و Redis، أصبح طلب إتمام الشراء يرجع للمستخدم بسرعة دون انتظار المهام الخلفية. أظهر الاختبار أن زمن استجابة الطلب كان 0.44 ثانية فقط، وكانت الفاتورة والإشعار غير جاهزين مباشرة بعد الطلب، مما يثبت أنهما لم ينفذا داخل المسار الرئيسي... بعد الانتظار لعدة ثوانٍ، تم توليد رابط الفاتورة وتحديث حالة الإشعار إلى sent بواسطة عامل Celery في الخلفية. هذا يثبت نجاح نقل المهام غير الضرورية للمستخدم إلى Queue مستقلة.

الطلب الرابع : اختبار Batch Processing

الهدف: معالجة مبيعات اليوم على دفعات Chunks بدل تحميل كل البيانات مرة واحدة.
❖ قبل تنفيذ الحل :

```
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> docker compose exec web1 python -m py
test tests/unit/test_batch_chunked.py::TestProcessInParallel::test_parallel_processing_completes_all_chunks -q
F [100%]
===== FAILURES =====
TestProcessInParallel.test_parallel_processing_completes_all_chunks
-----

# Define a simple handler that counts items
def count_handler(chunk):
    return {"count": len(chunk)}

qs = OrderItem.objects.filter(order__status=Order.PAID)
results = process_in_parallel(qs, count_handler, chunk_size=2, max_workers=3)

# Should have processed all 5 items across multiple chunks
total_count = sum(r["count"] for r in results)
assert total_count == 5
# Should have 3 chunks (2, 2, 1)
assert len(results) == 3
>
E   AssertionError: assert 1 == 3
E   + where 1 = len([{'count': 5}])

tests/unit/test_batch_chunked.py:426: AssertionError
----- Captured stderr call -----
batch.before_demo_materialized
----- Captured log call -----
INFO     core.batch:chunked.py:102 batch.before_demo_materialized
===== short test summary info =====
FAILED tests/unit/test_batch_chunked.py::TestProcessInParallel::test_parallel_processing_completes_all_chunks - Assertio
nError: assert 1 == 3
```

قبل تطبيق Batch Processing، لم يتم تقسيم البيانات إلى دفعات. عند وجود 5 عناصر وحجم دفعة 2 (chunk_size = 2)، كان المتوقع إنتاج 3 دفعات، لكن النظام أعاد نتيجة واحدة فقط، مما يعني أنه عالج كل البيانات دفعة واحدة. هذا يثبت غياب chunked processing واحتمال ارتفاع استهلاك الذاكرة عند التعامل مع بيانات ضخمة.

بعد تنفيذ الحل:

```
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> docker compose exec web1 python -m py
test tests/unit/test_batch_chunked.py -q
.....
11 passed in 1.31s [100%]
```

بعد تطبيق Batch Processing، نجحت جميع اختبارات المعالجة الدفعية وعددها 11 اختباراً. تثبت هذه الاختبارات أن النظام يقسم البيانات إلى Chunks، يعالجها، يدمج النتائج بشكل صحيح، وينشئ تقرير المبيعات اليومية بقيم صحيحة. النتيجة 11 passed تؤكد أن معالجة البيانات الضخمة على دفعات تعمل كما هو مطلوب.

الطلب الخامس : اختبار Load Distribution

الهدف: معرفة هل الطلبات تتوزع على عدة خوادم أم لا.
❖ قبل حل المشكلة :

```
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> $results = @{}
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> 1..20 | ForEach-Object {
>> $r = Invoke-WebRequest -Uri 'http://localhost/api/v1/instance/' -UseBasicParsing
>> $served = $r.Headers['X-Served-By']
>> if (-not $served) { $served = $r.Headers['X-Instance-Id'] }
>> if (-not $results.ContainsKey($served)) { $results[$served] = 0 }
>> $results[$served]++
>> }
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> $results.GetEnumerator() | Sort-Object Name | ForEach-Object {
>> "{0} {1}" -f $_.Name, $_.Value
>> }
172.18.0.4:8000 20
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine>
```

Nginx يوجه جميع الطلبات الى web1 فقط

```
12 upstream django_backend {
13     server web1:8000;
14 }
```

قبل تطبيق توزيع الأحمال، كان Nginx يمرر كل الطلبات إلى خادم واحد فقط. عند إرسال 20 طلباً، ظهرت جميعها على نفس الـ backend، مما يثبت عدم وجود Load Distribution. هذا يجعل web1 نقطة اختناق وفشل، بينما تبقى الخوادم الأخرى غير مستثمرة.

❖ بعد الحل :

```
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> $results = @{}
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> 1..30 | ForEach-Object {
>> $r = Invoke-WebRequest -Uri 'http://localhost/api/v1/instance/' -UseBasicParsing
>> $served = $r.Headers['X-Served-By']
>> if (-not $served) { $served = $r.Headers['X-Instance-Id'] }
>> if (-not $results.ContainsKey($served)) { $results[$served] = 0 }
>> $results[$served]++
>> }
PS C:\Users\Lenovo\Desktop\AI_4\تفرعية\High-Performance-E-Commerce-Backend-Engine> $results.GetEnumerator() | Sort-Object Name | ForEach-Object {
>> "{0} {1}" -f $_.Name, $_.Value
>> }
172.18.0.6:8000 10
172.18.0.8:8000 10
172.18.0.9:8000 10
```

بعد تطبيق توزيع الأحمال، تم إرسال 30 طلباً عبر Nginx إلى endpoint التشخيص. أظهرت النتائج أن الطلبات توزعت على أكثر من backend بدلاً من خادم واحد فقط، مما يثبت أن Nginx يعمل كـ Load Balancer ويوزع الحمل بين نسخ التطبيق web1, web2, و web3.